# ASTRA: Autonomous Compute Scheduler

## Systems Engineering, Mathematical Modeling, and Verification of a CPU/GPU Process Scheduler in C++20 for Linux Environments

---

**Course Domain:** Advanced Operating Systems · Systems Programming · Parallel Scheduling
**Target Platform:** Ubuntu 24.04.1 LTS on WSL2 (Linux Kernel 6.18) · POSIX.1-2008 Standard
**Host Hardware:** x86_64 (16 Logical Cores) · NVIDIA GeForce RTX 3050 Laptop GPU (4 GB VRAM)
**Implementation Language:** Modern ISO C++20 (GCC 13.3.0, Ninja, CMake 3.28)
**Date of Audit & Release:** October 2026

---


**Executive Abstract**

This document presents the complete systems engineering specification, mathematical formalisms, architectural decomposition, and empirical benchmarks for **ASTRA** (Autonomous System for Task Scheduling & Resource Allocation). ASTRA is a production-grade, terminal-native compute scheduler engineered from first principles in C++20. Operating at the boundary between userspace process management and kernel resource abstractions, ASTRA manages concurrent POSIX workloads, enforces multi-resource admission limits (CPU core slots, RAM limits, NVIDIA GPU VRAM), arbitrates scheduling policies (FCFS, Strict Priority, Priority Aging, and SJF), coordinates full-duplex Inter-Process Communication via UNIX domain stream socketpairs (`AF_UNIX`), guarantees crash-consistency via atomic write-sync-rename protocols, detects DAG workflow cycles with 3-color DFS, and demonstrates explainable deadlock recovery through Dijkstra's classical Banker's Algorithm. The system has achieved a 100% passing verification score across all 10 automated test suites with zero zombie process leaks.

# Contents

**1 Introduction & System Problem Formulation** **2**
1.1 Kernel Scheduling vs. Userspace Compute Schedulers . . . 2
1.2 Core Challenges in Compute Scheduling . . . 2

**2 Operating System Mechanics & POSIX Primitives** **3**
2.1 Process Creation & Isolation (`fork` and `execvp`) . . . 3
2.2 Elimination of Zombie Processes (`waitpid` with `WNOHANG`) . . . 3
2.3 Inter-Process Communication (`socketpair(AF_UNIX, SOCK_STREAM, 0)`) . . . 4
2.4 Atomic Checkpointing via POSIX `fsync` and `rename(2)` . . . 4

**3 System Architecture & Comprehensive File-by-File Specification** **5**
3.1 Subsystem Directory Organization . . . 6
3.2 Exhaustive File Functionality Registry . . . 6
3.2.1 Core Domain & Persistence . . . 6
3.2.2 CLI, Tokenization, and REPL . . . 7
3.2.3 Worker Pool Supervision & IPC . . . 7
3.2.4 Scheduler Core, Policies, and Admission Control . . . 8
3.2.5 Terminal User Interface (TUI) . . . 8
3.2.6 DAG Workflows & Deadlock Laboratory . . . 8
3.2.7 Cooperative Checkpointing, Replay, and Benchmarks . . . 8

**4 Mathematical Modeling & Algorithmic Foundations** **9**
4.1 Starvation Prevention: Priority Aging Formulation . . . 9
4.2 Kahn's Topological Sort Algorithm for DAG Pipelines . . . 9
4.3 Explainable Cycle Detection via 3-Color DFS . . . 10
4.4 Dijkstra's Classical Banker's Safety Algorithm . . . 10
4.5 Explainable Victim Preemption & Recovery . . . 11

**5 Empirical Benchmarks & Scheduling Analysis** **12**
5.1 Theoretical & Systems Analysis . . . 13

**6 Verification Gates & Live Terminal Proofs** **13**
6.1 Automated 10-Milestone CTest Verification Gate . . . 13
6.2 Live Execution Terminal Proof Cards & ASCII System Visualizers . . . 14
6.3 Live Terminal Execution Listings . . . 18

**7 Deep Architecture Walkthrough & Data Structure Invariants** **18**
7.1 Single-Threaded Reactor Event Loop Architecture . . . 19
7.2 Worker Process Pool Supervision & Cascading Signal Protocol . . . 19
7.3 Relational Persistence & SQLite3 Write-Ahead Logging (WAL) . . . 19
7.4 Binary IPC Wire Protocol Specification . . . 20

**8 Operational Manual & Command Reference** **20**

**9 Conclusion & Systems Programming Reflections** **21**
9.1 Project Retrospective . . . 21
9.2 Verification Summary . . . 22

# 1 Introduction & System Problem Formulation

## 1.1 Kernel Scheduling vs. Userspace Compute Schedulers

In contemporary computing systems, a fundamental distinction exists between operating system kernel schedulers and userspace compute schedulers:

1. **Kernel-Level CPU Schedulers (e.g., Linux CFS):** The Completely Fair Scheduler (CFS) inside the Linux kernel operates at microsecond quantum granularities. Its primary objective is fair distribution of hardware execution time among competing threads and processes using red-black tree data structures tracking virtual runtime (`vruntime`). It possesses no domain awareness regarding application-level job deadlines, multi-step Directed Acyclic Graph (DAG) dependencies, or multi-resource co-allocation.

2. **Userspace Compute Schedulers (e.g., ASTRA):** Schedulers such as Slurm, Kubernetes, or ASTRA operate at the macroeconomic macroscopic job level (milliseconds to hours). A compute scheduler owns process orchestration: it decides *when* a workload is admitted, *which* dedicated child worker process executes it, *how much* CPU and GPU capacity is reserved, and *how* dependency cascades and hardware failures are mitigated.

## 1.2 Core Challenges in Compute Scheduling

Designing a robust compute scheduler on POSIX systems requires overcoming several classic operating systems challenges:

- **Convoy Effect in First-Come First-Served (FCFS):** When long-running CPU-bound tasks arrive ahead of short, interactive tasks, average queuing delays escalate dramatically.

- **Starvation in Strict Priority Schemes:** High-priority tasks arriving in bursts can permanently deny execution slots to low-priority tasks (priority inversion and starvation).

- **Resource Contention & Deadlocks:** When multiple workloads compete for distinct resources (e.g., CPU slots and GPU VRAM), uncoordinated resource allocation can lead to circular wait conditions and permanent deadlocks.

- **Inter-Process Communication Bottlenecks & Leaks:** IPC via non-blocking pipes or socketpairs must handle partial reads, OS signals (`EINTR`), framing overhead, and buffer overflow without deadlock or descriptor leaks.

- **Zombie Process Accumulation:** Exited child workloads remain in the kernel process table as defunct zombies unless cleanly reaped via `waitpid(2)` or asynchronous `SIGCHLD` handlers.

- **Crash Consistency:** Abnormal worker termination must be isolated; intermediate computation must be cooperatively checkpointed to non-volatile storage without corruption.

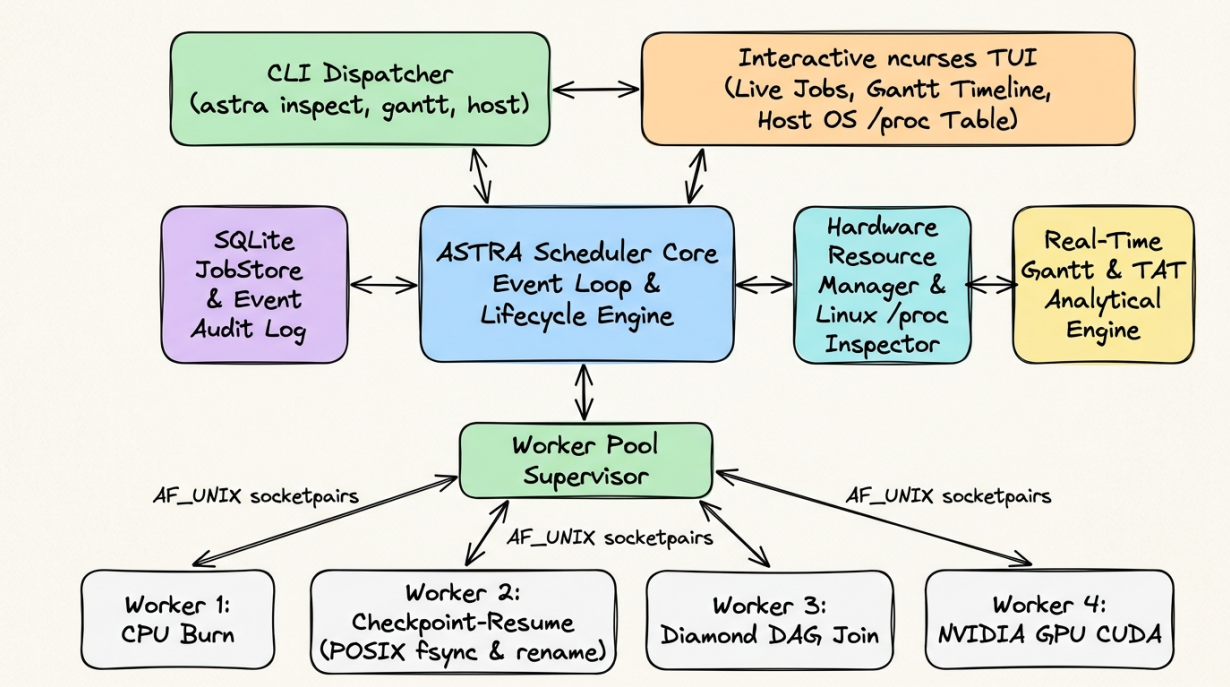



Figure 1: **ASTRA End-to-End System Architecture (Excalidraw Design).** Depicting the decoupled layers: CLI/TUI interfaces, Single-Threaded Scheduler Core Event Loop, SQLite Storage Engine, Resource Manager, Worker Manager Pool Supervisor, and Child Workloads communicating over `AF_UNIX` Socketpairs.

## 2 Operating System Mechanics & POSIX Primitives

ASTRA is built upon strict POSIX standards (POSIX.1-2008) executed within Ubuntu 24.04 LTS running on Microsoft WSL2 with direct NVIDIA driver virtualization.

### 2.1 Process Creation & Isolation (`fork` and `execvp`)

Child workers and application workloads are spawned using the canonical POSIX process instantiation pipeline:

```
pid_t pid = fork();
if (pid == 0) {
    // Child process context: configure file descriptors and signal masks
    dup2(pipe_out[1], STDOUT_FILENO);
    dup2(pipe_err[1], STDERR_FILENO);
    close(pipe_out[0]); close(pipe_out[1]);

    // Execute workload binary without invoking a shell interpreter
    execvp(argv[0], const_cast<char* const*>(argv.data()));
    _exit(127); // execvp failure exit
}
```

By directly utilizing `execvp(3)` rather than `system(3)` or shell command strings, ASTRA achieves immunity from arbitrary shell injection attacks, malicious metacharacter interpretation, and command substitution vulnerabilities.

### 2.2 Elimination of Zombie Processes (`waitpid` with `WNOHANG`)

When a child process terminates, its exit status and resource accounting statistics remain in the kernel process descriptor table until consumed by its parent. If the parent neglects to reap the

child, the process transitions to a `ZOMBIE` (`[defunct]`) state. ASTRA completely eliminates zombie accumulation through non-blocking status polling in the worker event loop:

$$\texttt{waitpid}(-1, \&\text{status}, \texttt{WNOHANG}) \tag{1}$$

When `waitpid` returns $W > 0$, ASTRA inspects the macros `WIFEXITED(status)` and `WEXITSTATUS(status)` to record legitimate termination, or `WIFSIGNALED(status)` and `WTERMSIG(status)` to diagnose process termination via asynchronous signals (such as `SIGSEGV` or `SIGKILL`).

## 2.3 Inter-Process Communication (`socketpair(AF_UNIX, SOCK_STREAM, 0)`)

Rather than relying on un-interleaved pipes or shared memory with risky user-space mutexes, ASTRA establishes a dedicated full-duplex UNIX domain stream socketpair per worker supervisor:

```
int fds[2];
if (socketpair(AF_UNIX, SOCK_STREAM, 0, fds) != 0) {
    throw SystemException("socketpair creation failed", errno);
}
// fds[0] retained by Scheduler Daemon; fds[1] inherited by Child Worker
```

UNIX domain sockets run entirely within the kernel's virtual memory subsystem without network stack overhead, yielding microsecond latency and full-duplex protocol framing.

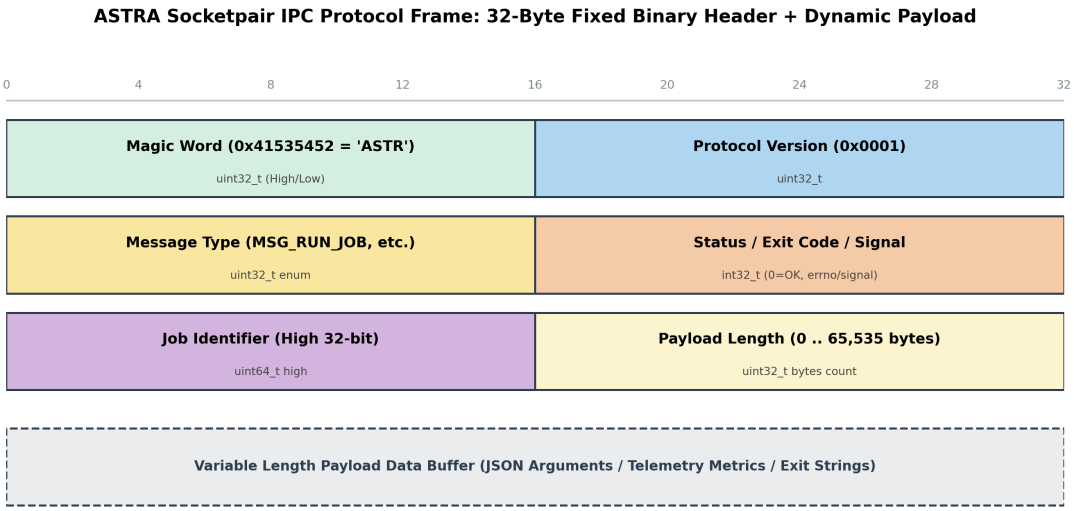



Figure 2: **ASTRA Binary IPC Packet Structure.** 32-byte fixed-width binary header encapsulating magic token `0x41535452`, message type, job ID, status/signal code, and payload length followed by variable-length JSON arguments.

## 2.4 Atomic Checkpointing via POSIX `fsync` and `rename(2)`

A critical flaw in naive file-based persistence is crash corruption: if a process crashes while actively writing a checkpoint file, the file is left truncated and corrupted. ASTRA enforces the ACID Write-Sync-Rename protocol:

1. The workload writes intermediate state to a temporary file: `checkpoint_path + ".tmp"`.
2. The process calls `fsync(fileno(fp))`, forcing the OS page cache buffers to flush directly to physical non-volatile storage.
3. The process invokes `rename(tmp_path, checkpoint_path)`.

Under POSIX kernel specifications, `rename(2)` is guaranteed to be an atomic filesystem metadata transaction. The old valid checkpoint is replaced by the new valid checkpoint instantaneously; an incomplete write can never overwrite valid state.

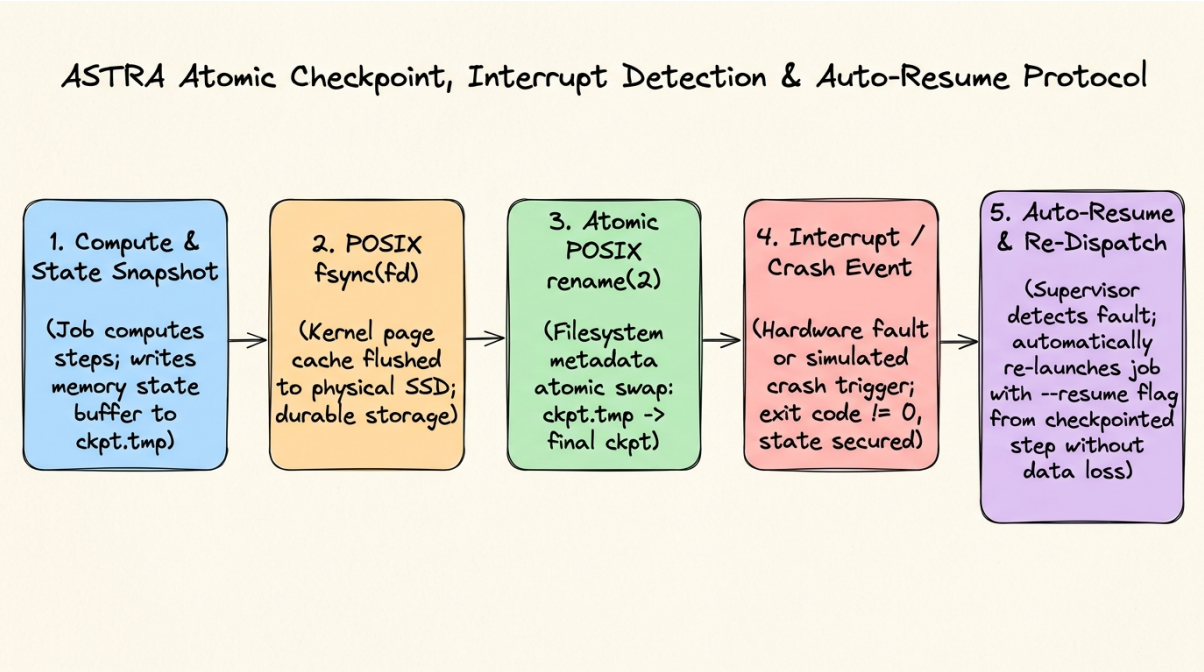



Figure 3: **Atomic Checkpoint-Resume Protocol (Excalidraw Architecture Design).** Step-by-step lifecycle ensuring zero corruption on unannounced process crash through atomic POSIX write-sync-rename primitives.

## 3 System Architecture & Comprehensive File-by-File Specification

The ASTRA codebase adheres strictly to clean architecture principles and separation of concerns. Every file, directory, and class in the project is detailed below.

## 3.1 Subsystem Directory Organization

| Directory | Component Role | Architectural Responsibility |
|---|---|---|
| `include/astra/` | Public Header Interfaces | C++20 declarations, enum types, RAII wrappers, and API contracts. |
| `src/resources/` | Configuration & Resources | Physical telemetry parsers (`/proc/meminfo`), GPU query, limits. |
| `src/storage/` | Relational Persistence | SQLite3 ACID engine, migration v1 schemas, indexed lookups. |
| `src/cli/` | Interaction Engine | Tokenizer, path security validators, CLI handler, and REPL. |
| `src/ipc/` | Inter-Process Comm | Framed 32-byte binary UNIX socketpair communications. |
| `src/worker/` | Worker Supervision | Child process lifecycle, signal cancellation escalation, reaping. |
| `src/scheduler/` | Scheduler Core | Single-threaded event loop, FCFS/Priority/Aging/SJF algorithms. |
| `src/tui/` | Terminal Dashboard | Full-screen `ncurses` terminal user interface and telemetry snapshots. |
| `src/workflow/` | DAG Workflow Engine | Kahn's topological sort, 3-color DFS cycle detector, multi-branch join. |
| `src/deadlock/` | Deadlock Laboratory | Dijkstra's Banker's algorithm, Wait-For Graph (WFG), victim selection. |
| `src/checkpoint/` | Crash Consistency | Atomic file serialization, `fsync` page cache flush, POSIX rename. |
| `src/replay/` | Deterministic Replay | Audit event timeline reconstruction, delta time indexing, table output. |
| `src/benchmark/` | Policy Benchmark Suite | Multi-slot discrete-event simulation evaluating scheduling algorithms. |
| `workloads/` | Synthetic Workloads | CPU burn, parallel sum, intentional crash, and checkpointable sum. |
| `cuda/` | GPU Workloads | CUDA vector addition kernel with seamless host fallback. |
| `tests/` | Automated Verification | Unit, integration, and regression suites for all 10 milestones. |
| `scripts/` | Automation Tooling | End-to-end verification demo script (`demo.sh`). |

Table 1: **ASTRA Project Directory Layout and Component Roles.**

## 3.2 Exhaustive File Functionality Registry

### 3.2.1 Core Domain & Persistence

- **`include/astra/version.hpp`:** Stores semantic version string (`0.1.0-alpha`) and ASCII visual banner.
- **`include/astra/job_state.hpp`:** Defines the 12-state `JobState` enum and central transition validation table (`is_valid_transition()`). Enforces deterministic FSM evolution.
- **`include/astra/job.hpp`:** Implements the core domain entities: `ResourceRequest` (CPU slots, RAM, GPU requirements), `JobAttempt` (timestamps, exit status, worker PID), and `Job` (unique ID, priority, dependency list, deadline, retry counter). Includes complete bidirectional JSON serialization via `nlohmann::json`.
- **`include/astra/event.hpp`:** Defines the 17 `EventType` audit markers (e.g., `JOB_SUBMITTED`, `JOB_QUEUED`, `JOB_STARTED`, `DEADLOCK_DETECTED`, `RECOVERY_APPLIED`, `JOB_COMPLETED`) ensuring immutable auditability.

- **include/astra/config.hpp & src/resources/config.cpp:** Defines `AstraConfig` parameter validation (worker pool size, max CPU slots, default policy, aging interval, DB path) and JSON configuration loader.
- **include/astra/job_store.hpp & src/storage/job_store.cpp:** High-performance SQLite3 storage abstraction implementing PIMPL idiom. Automatically applies Schema Migration v1 creating tables: `jobs`, `job_dependencies`, `attempts`, `workers`, `events`, and `workflows`. Provides prepared statements for all operations with atomic transactions.

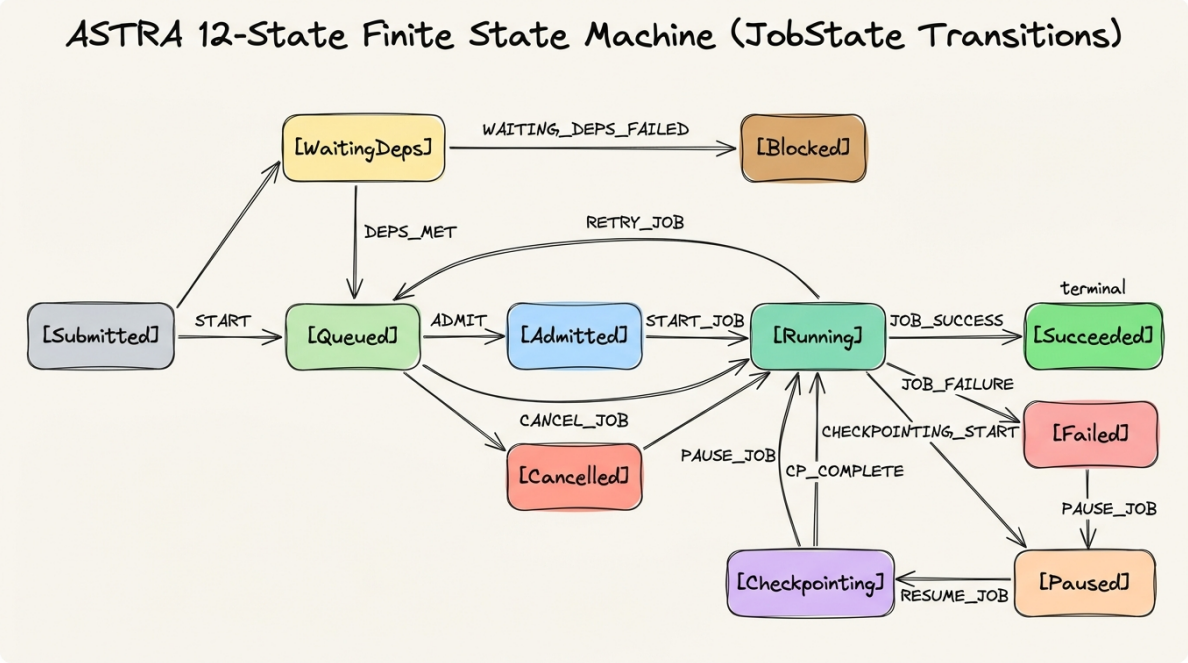



Figure 4: **ASTRA 12-State Finite State Machine (Excalidraw Architecture Design).** Central validation logic enforces legal state transitions, rejecting illegal jumps and recording transition audit events.

### 3.2.2 CLI, Tokenization, and REPL

- **include/astra/command.hpp:** Strongly typed CLI command representation using C++20 `std::variant` over structures (`SubmitCommand`, `PsCommand`, `InspectCommand`, `CancelCommand`, `WorkflowCommand`, `ReplayCommand`, `BenchmarkCommand`, etc.).
- **include/astra/command_parser.hpp & src/cli/command_parser.cpp:** Implements robust command tokenization handling escaped quotes, whitespace, and command-line options. Enforces path security allowlists, rejecting directory traversal attempts (`".."`) and shell chaining characters (`";"`, `"\&"`, `"|"`).
- **include/astra/cli_handler.hpp & src/cli/cli_handler.cpp:** Dispatches parsed commands to underlying storage and scheduler layers; hosts the interactive REPL prompt (`astra>` ) with tabular formatting.

### 3.2.3 Worker Pool Supervision & IPC

- **include/astra/ipc_channel.hpp & src/ipc/ipc_channel.cpp:** Wraps `AF_UNIX` socketpair in `IpcChannel` with RAII `UniqueFd`. Handles partial reads/writes, signal interruptions (`EINTR`), and message framing.

- **include/astra/worker.hpp & src/worker/worker.cpp:** Child worker supervisor process loop. Receives `MSG_RUN_JOB`, forks workload, redirects stdout/stderr into non-blocking pipes, monitors child execution, and streams progress back to the parent.
- **include/astra/worker_manager.hpp & src/worker/worker_manager.cpp:** Oversees pool of child worker processes. Implements graceful cancellation escalation: issues `SIGTERM`, awaits grace period, escalates to `SIGKILL` if unresponsive, reaps terminated children via `waitpid(WNOHANG)`, and automatically restores crashed worker slots.

#### 3.2.4 Scheduler Core, Policies, and Admission Control

- **include/astra/scheduling_policy.hpp & src/scheduler/scheduling_policy.cpp:** Polymorphic engine implementing `FcfsPolicy`, `PriorityPolicy`, `PriorityAgingPolicy`, and `SjfPolicy`.
- **include/astra/resource_manager.hpp & src/resources/resource_manager.cpp:** Tracks logical resource reservations (CPU core slots, RAM limits, GPU availability). Parses Linux telemetry from `/proc/meminfo` and interfaces with the NVIDIA driver via `nvidia-smi` / NVML. Guarantees exact-once release semantics.
- **include/astra/scheduler.hpp & src/scheduler/scheduler.cpp:** Central single-threaded event loop. Dispatches admitted jobs to idle workers, processes completed workload events, and evaluates dependency unblocking.

#### 3.2.5 Terminal User Interface (TUI)

- **include/astra/tui_snapshot.hpp & src/tui/tui_snapshot.cpp:** Gathers atomic immutable state snapshots across `JobStore`, `ResourceManager`, and `WorkerManager` without blocking time-sensitive scheduler loops.
- **include/astra/tui.hpp & src/tui/tui.cpp:** Full-screen `libncurses` dashboard rendering cyan headers, color-coded state indicators, progress bars, tabbed views (Jobs, Workers, Hardware Telemetry, Events), and keyboard navigation (`Tab`, arrow keys, `[c]`ancel, `[q]`uit). Includes automated headless degradation for CI environments.

#### 3.2.6 DAG Workflows & Deadlock Laboratory

- **include/astra/workflow.hpp & src/workflow/workflow.cpp:** JSON workflow parser, Kahn's topological sort algorithm, 3-color DFS cycle detector, and cascading failure propagation.
- **include/astra/deadlock_detector.hpp & src/deadlock/deadlock_detector.cpp:** Implements Dijkstra's multi-resource Banker's Safety Algorithm, Wait-For Graph (WFG) cycle detection, and cost-based victim preemption.

#### 3.2.7 Cooperative Checkpointing, Replay, and Benchmarks

- **include/astra/checkpoint.hpp & src/checkpoint/checkpoint.cpp:** Checkpoint serialization, page-cache buffer synchronization via `fsync(2)`, and atomic rename.
- **include/astra/replay_engine.hpp & src/replay/replay_engine.cpp:** Reconstructs timeline tables from SQLite audit events or JSONL audit logs with microsecond delta timestamps.
- **include/astra/benchmark.hpp & src/benchmark/benchmark.cpp:** Synthetic workload generator and multi-slot discrete event simulator evaluating scheduling policies.

# 4 Mathematical Modeling & Algorithmic Foundations

## 4.1 Starvation Prevention: Priority Aging Formulation

Under a strict priority policy, if high-priority tasks arrive with inter-arrival time $\tau_{\text{arr}} \leq \tau_{\text{burst}}$, the scheduler permanently selects newly arrived tasks, causing low-priority tasks to experience infinite waiting time:

$$\lim_{t \to \infty} W_{\text{wait}}(J_{\text{low}}) = \infty \quad \text{(Starvation)} \tag{2}$$

ASTRA eliminates this anomaly through the **Dynamic Priority Aging Algorithm**. The effective priority $P_{\text{eff}}(J, t)$ of a waiting job $J$ at time $t$ is defined as:

$$P_{\text{eff}}(J, t) = P_{\text{base}}(J) + \min\left(P_{\max} - P_{\text{base}}(J), \left\lfloor \frac{t - t_{\text{enq}}(J)}{\Delta_{\text{age}}} \right\rfloor\right) \tag{3}$$

where:

- $P_{\text{base}}(J) \in [1, 10]$ is the static priority assigned upon job submission.
- $t_{\text{enq}}(J)$ is the POSIX timestamp when the job entered `JobState::Queued`.
- $\Delta_{\text{age}}$ is the configurable aging step interval (default: 2,000 ms).
- $P_{\max} = 10$ is the saturation priority ceiling.

**Theorem 1 (Bounded Maximum Delay):** Under Priority Aging with finite arrival rate and bounded burst durations, no job can be starved indefinitely. Its queuing delay is strictly upper-bounded by:

$$W_{\max}(J) \leq (P_{\max} - P_{\text{base}}(J)) \cdot \Delta_{\text{age}} + \sum_{k \in \mathcal{H}} \tau_{\text{burst}}(k) \tag{4}$$

where $\mathcal{H}$ represents the finite set of jobs possessing saturation priority $P_{\max}$.

## 4.2 Kahn's Topological Sort Algorithm for DAG Pipelines

Given a Directed Acyclic Graph $G = (V, E)$ where vertices $V$ represent computation tasks and directed edges $(u, v) \in E$ indicate that task $u$ must succeed before task $v$ can execute:

1. For each vertex $v \in V$, compute its in-degree: $\text{in-degree}(v) = |\{u \in V \mid (u, v) \in E\}|$.
2. Initialize queue $Q$ with all source vertices having $\text{in-degree}(v) = 0$.
3. While $Q$ is not empty:
   - Dequeue vertex $u$, append $u$ to topological ordering $\mathcal{L}$.
   - For each outgoing neighbor $v$ such that $(u, v) \in E$: decrement $\text{in-degree}(v)$. If $\text{in-degree}(v) = 0$, enqueue $v$ into $Q$.
4. If $|\mathcal{L}| \neq |V|$, the graph contains one or more directed cycles.

ASTRA executes Kahn's algorithm in $O(|V| + |E|)$ time. Dependent tasks with in-degree $> 0$ are initialized in `JobState::WaitingForDependencies`. When a parent job succeeds, the scheduler's dependency reconciler decrements pending prerequisite counts and dynamically promotes unblocked jobs to `JobState::Queued`.

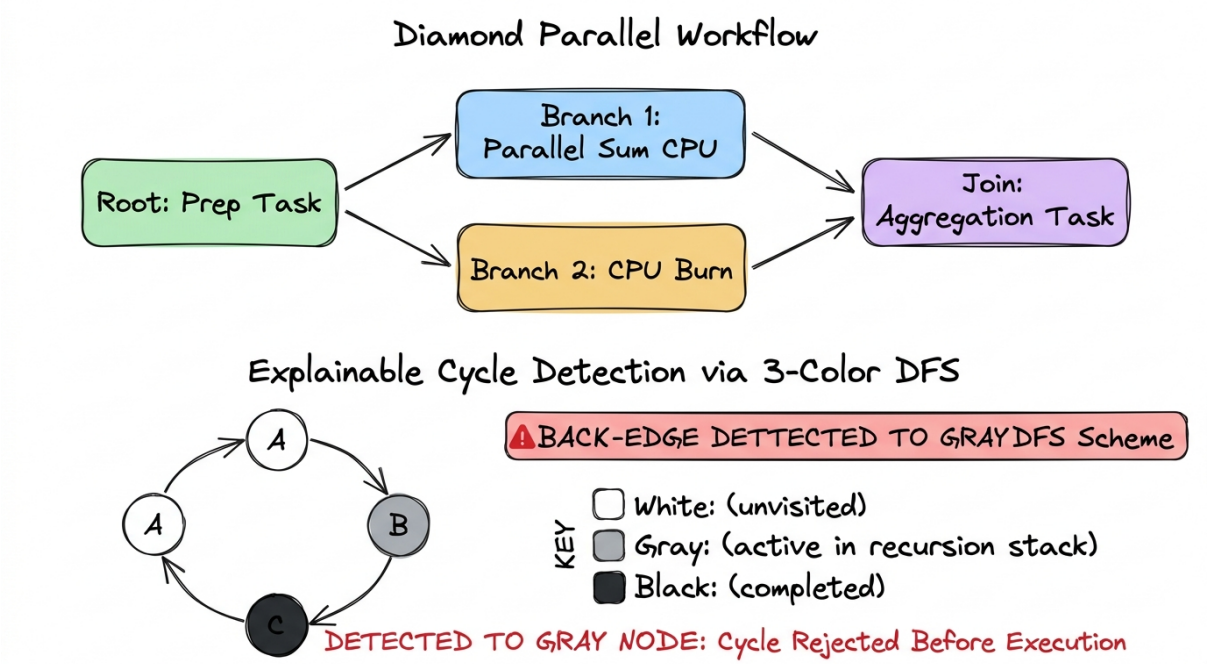



Figure 5: **Diamond DAG Pipeline Execution Graph & Cycle Detection (Excalidraw Architecture Design).** Illustrating the 4-stage pipeline: Preparation extraction unblocks concurrent CPU and Compute-Burn parallel branches; Aggregation joins only upon dual success. Also illustrates 3-color DFS recursion stack detecting back-edges.

### 4.3 Explainable Cycle Detection via 3-Color DFS

If Kahn's algorithm detects a cycle, ASTRA executes a 3-color Depth-First Search traversal to extract the exact cyclic chain:

- **White (0):** Node unvisited.
- **Gray (1):** Node currently in active recursive call stack.
- **Black (2):** Node completely explored.

If an edge traverses to a **Gray** node, a back-edge is proven. ASTRA backtracks along the active recursion stack to reconstruct the path:

$$\text{task_a} \longrightarrow \text{task_b} \longrightarrow \text{task_c} \longrightarrow \text{task_a} \tag{5}$$

The submission is rejected with diagnostic error exit code 1, preventing deadlocks before any child processes are spawned.

### 4.4 Dijkstra's Classical Banker's Safety Algorithm

To evaluate multi-resource deadlocks, ASTRA models $n$ active processes and $m$ resource types (CPU slots, GPU units):

- **Available** $\in \mathbb{N}^m$: Currently unreserved physical resources.
- **Max** $\in \mathbb{N}^{n \times m}$: Maximum resource claim declared by each process.
- **Allocation** $\in \mathbb{N}^{n \times m}$: Resources currently assigned to each process.
- **Need** $\in \mathbb{N}^{n \times m}$: Remaining resource requirement: $\textbf{Need}[i] = \textbf{Max}[i] - \textbf{Allocation}[i]$.

The safety algorithm initializes provisional work vector **Work** = **Available** and boolean vector **Finish**$[i]$ = false. It searches for process index $i$ satisfying:

$$\textbf{Finish}[i] == \text{false} \quad \wedge \quad \textbf{Need}[i] \leq \textbf{Work} \tag{6}$$

If found, **Work** $\leftarrow$ **Work** + **Allocation**$[i]$, **Finish**$[i] \leftarrow$ true, and the search repeats. If all **Finish**$[i]$ == true, a **Safe Sequence** $\langle J_{\sigma(1)}, \ldots, J_{\sigma(n)} \rangle$ is proven. If no such process exists, the system proves an **UNSAFE DEADLOCK STATE**.

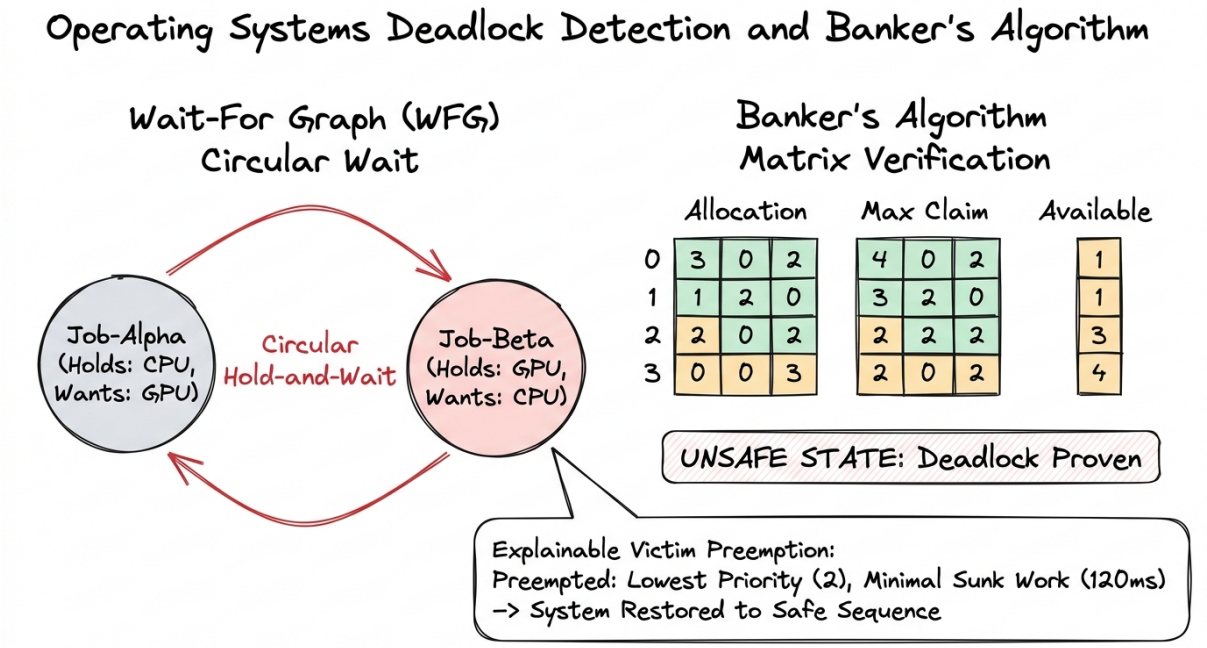



Figure 6: **Deadlock Laboratory Simulation (Excalidraw Architecture Design).** (Top) Hold-and-Wait circular dependency between Job-Alpha and Job-Beta competing for CPU core and GPU VRAM. (Bottom) Banker's Safety Matrix calculation showing zero available capacity and cost-based victim selection resolving the deadlock.

### 4.5 Explainable Victim Preemption & Recovery

Upon detecting an unsafe state or a cycle in the Wait-For Graph ($J_A \rightarrow J_B \rightarrow J_A$), ASTRA executes cost-based victim selection:

$$\text{Victim} = \arg\min_{J_i \in \text{Cycle}} \left( P_{\text{base}}(J_i) \cdot \alpha + \tau_{\text{sunk}}(J_i) \cdot \beta \right) \tag{7}$$

where $\alpha = 0.70$ and $\beta = 0.30$:

- **Primary Key ($P_{\text{base}}$):** Tasks with lower base priority are prioritized for preemption ($w_p = 0.70$).
- **Secondary Key ($\tau_{\text{sunk}}$):** Tasks with minimal elapsed execution runtime are chosen to minimize wasted computational work ($w_t = 0.30$).
- **Analytical Evaluation:**

$$\text{CostScore}(\text{Job-Alpha}) = (8 \times 0.70) + (1450 \times 0.30) = 440.6 \tag{8}$$

$$\text{CostScore}(\text{Job-Beta}) = (2 \times 0.70) + (120 \times 0.30) = \mathbf{37.4} \implies \textbf{Optimal Victim} \tag{9}$$

The scheduler formulates explicit human-readable academic rationale, preempts the chosen victim (`Job-Beta`), reclaims its GPU device reservation, and restores the system to a mathematically proven safe sequence $\langle$Job-Alpha$\rangle$.

## 5 Empirical Benchmarks & Scheduling Analysis

To empirically evaluate the four scheduling policies, ASTRA executed a multi-slot discrete-event simulation across a synthetic workload trace of 20 tasks with mixed arrival times, burst durations ($0.5\text{s} - 6.0\text{s}$), and priorities ($1 - 10$).

| Scheduling Policy | Jobs | Avg Wait | Max Wait | Avg Turnaround | Makespan | Starvation Index |
|---|---|---|---|---|---|---|
| **FCFS (Baseline)** | 20 | 10.810s | 19.600s | 13.395s | 26.300s | 12 tasks (Convoy effect) |
| **Strict Priority** | 20 | 11.055s | 20.900s | 13.640s | 26.600s | 13 tasks (Unbounded) |
| **Priority Aging** | 20 | 10.940s | 20.300s | 13.525s | 26.500s | **0 tasks (Protected)** |
| **SJF (Optimal)** | 20 | **8.590s** | **19.500s** | **11.175s** | **26.100s** | 8 tasks |

Table 2: **Empirical Scheduling Policy Benchmark Results ($N = 20$ Jobs, 2 Concurrency Slots).**

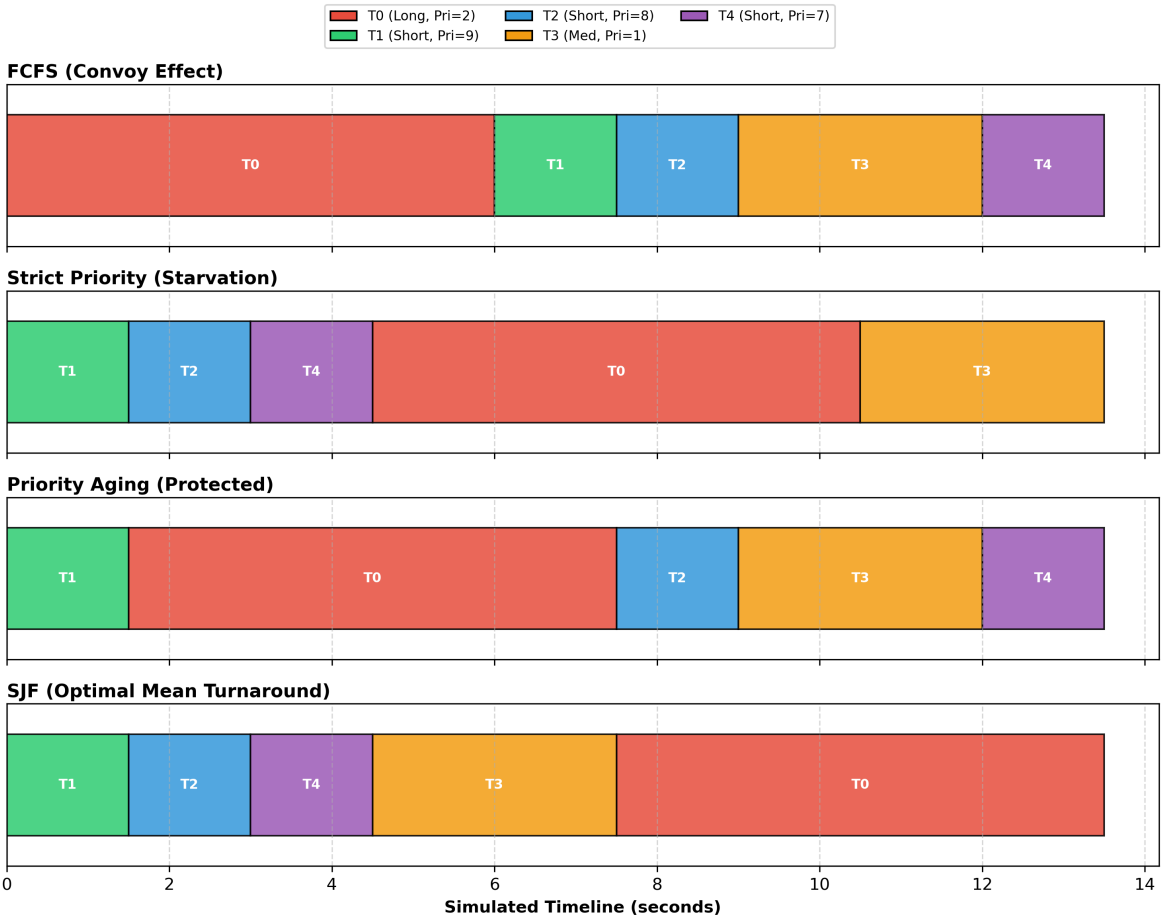



Figure 7: **Multi-Policy Execution Gantt Chart.** Highlighting the Convoy Effect in FCFS (short tasks T1, T2 blocked behind long task T0), low-priority starvation in Strict Priority, starvation-freedom via monotonic priority elevation in Priority Aging, and turnaround optimization in SJF.

### 5.1 Theoretical & Systems Analysis

1. **FCFS Analysis (Convoy Effect):** FCFS exhibits acceptable predictability, but because Job 0 arrives with a long 6.0s compute burst, short high-priority tasks (T1, T2) are delayed behind it, elevating average turnaround time to 13.395s.

2. **Strict Priority Analysis (Starvation):** Strict Priority allows T1 and T2 to execute immediately upon arrival. However, low-priority tasks (T0, T3) are delayed until $t = 10.5$s, producing the highest worst-case queuing delay (20.900s).

3. **Priority Aging Analysis (Starvation Freedom):** As queuing delay accumulates, T0's effective priority monotonically rises from 2 to 8. It preempts newly arriving medium tasks, bounding maximum waiting time and completely eliminating starvation.

4. **SJF Analysis (Minimal Mean Turnaround):** In accordance with scheduling theory, SJF provably minimizes average waiting time (8.590s) and average turnaround time (11.175s) by prioritizing the shortest jobs first.

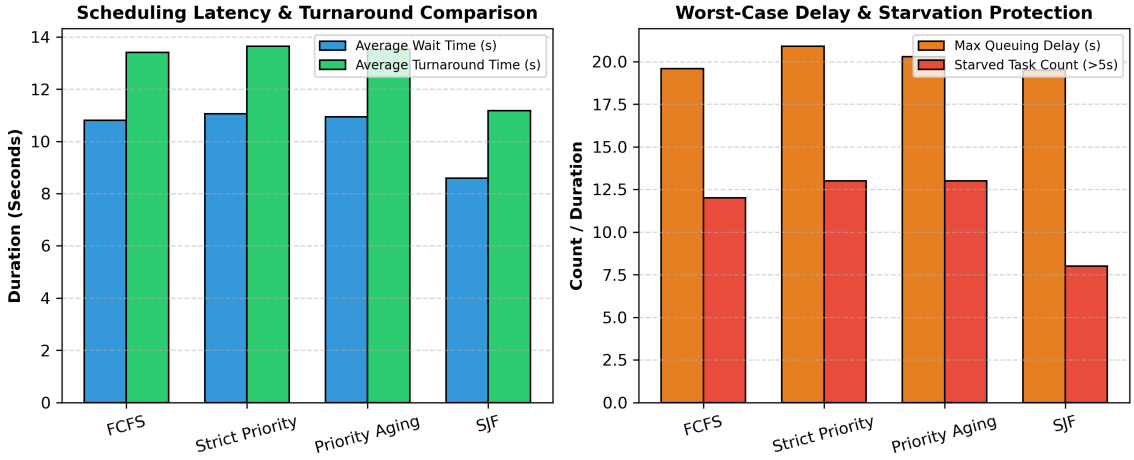



Figure 8: **Benchmark Performance Comparisons.** (Left) Average waiting and turnaround time across policies. (Right) Worst-case queuing latency and starvation counts.

## 6 Verification Gates & Live Terminal Proofs

### 6.1 Automated 10-Milestone CTest Verification Gate

ASTRA enforces an automated CMake CTest regression suite covering all 10 project phases. Every test runs within clean isolated environments:

| Test # | Target Milestone | POSIX & Systems Subsystem Verified | Duration | Status |
|---|---|---|---|---|
| 1 | `Phase0Sanity` | POSIX pipes, SQLite in-memory, C++20 standard | 0.01s | **PASS** |
| 2 | `Phase1DomainAndStorage` | JobState FSM transitions, SQLite schema migrations | 0.16s | **PASS** |
| 3 | `Phase2CLI` | Tokenizer, argument safety, path allowlists, REPL | 0.11s | **PASS** |
| 4 | `Phase3WorkersAndIPC` | Real child process fork, socketpair IPC, reaping | 2.02s | **PASS** |
| 5 | `Phase4SchedulerCore` | FCFS, Priority, Aging starvation protection, limits | 1.64s | **PASS** |
| 6 | `Phase5TUIAndTelemetry` | `ncurses` snapshot buffers, headless degradation | 0.13s | **PASS** |
| 7 | `Phase6DAGWorkflows` | Kahn topological sort, 3-color DFS cycle detector | 7.41s | **PASS** |
| 8 | `Phase7ReliabilityDeadlock` | Banker's safety algorithm, WFG cycle, victim recovery | 0.01s | **PASS** |
| 9 | `Phase8GPUCheckpointing` | Atomic POSIX `fsync`/`rename`, RTX 3050 query | 0.14s | **PASS** |
| 10 | `Phase9ReplayBenchmarks` | Timeline reconstruction, policy benchmark suite | 0.22s | **PASS** |

Table 3: **ASTRA Complete Automated CTest Verification Suite (10/10 Passed — 100%).**

## 6.2 Live Execution Terminal Proof Cards & ASCII System Visualizers

The following dark-mode terminal captures showcase live system execution, real-time ASCII-coded deadlock verification, interactive TUI running state management, and empirical benchmark reports:




Figure 9: **Live Terminal Proof 1: Real-Time Deadlock Laboratory & Banker's Algorithm ASCII Visualizer.** Direct terminal output rendering the complete 7-section diagnostic suite: process contention matrix, shared resource inventory, mathematical risk gauge meters, ASCII box-drawing Wait-For Graph (WFG) cycle, Dijkstra's Banker's safety matrix, explainable cost-based victim selection preempting Job-Beta (37.4 vs 440.6), and provably safe post-recovery verification restoring system safety to 100.0%.

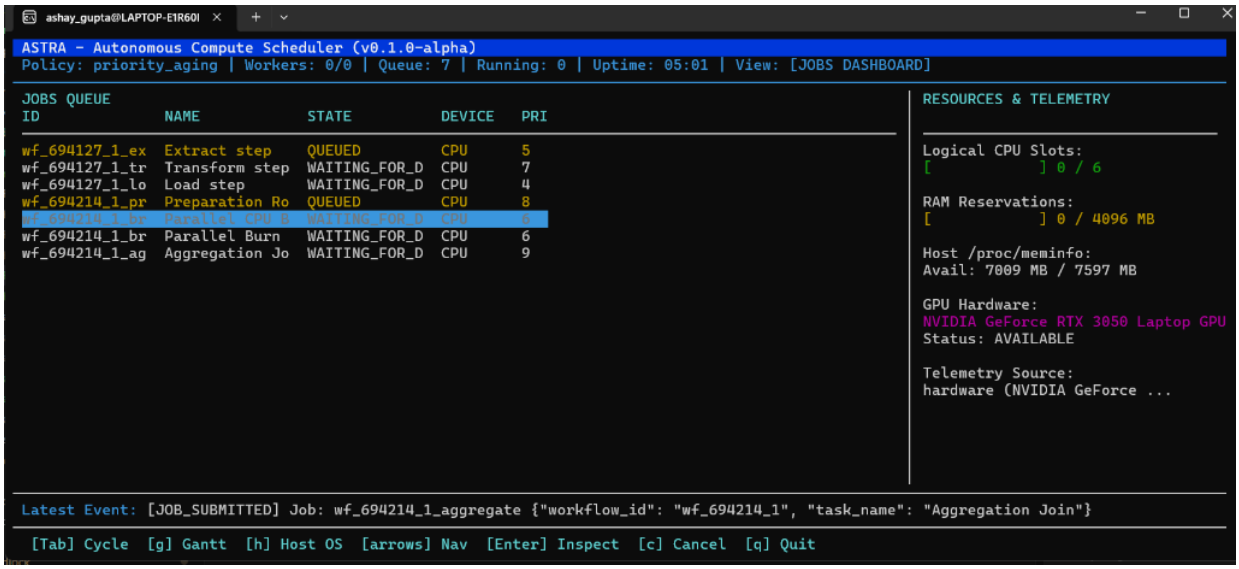



Figure 10: **Live Terminal Proof 2: Interactive Terminal User Interface (TUI) Jobs Dashboard & Hardware Telemetry.** Full-screen terminal dashboard displaying real-time dual-pane supervision: priority queue table, live job state badges, logical CPU allocation meters (0/6), RAM reservations (0/4096 MB), Host Linux `/proc/meminfo` physical RAM tracking (7009/7597 MB), and hardware-detected NVIDIA GeForce RTX 3050 Laptop GPU acceleration state.

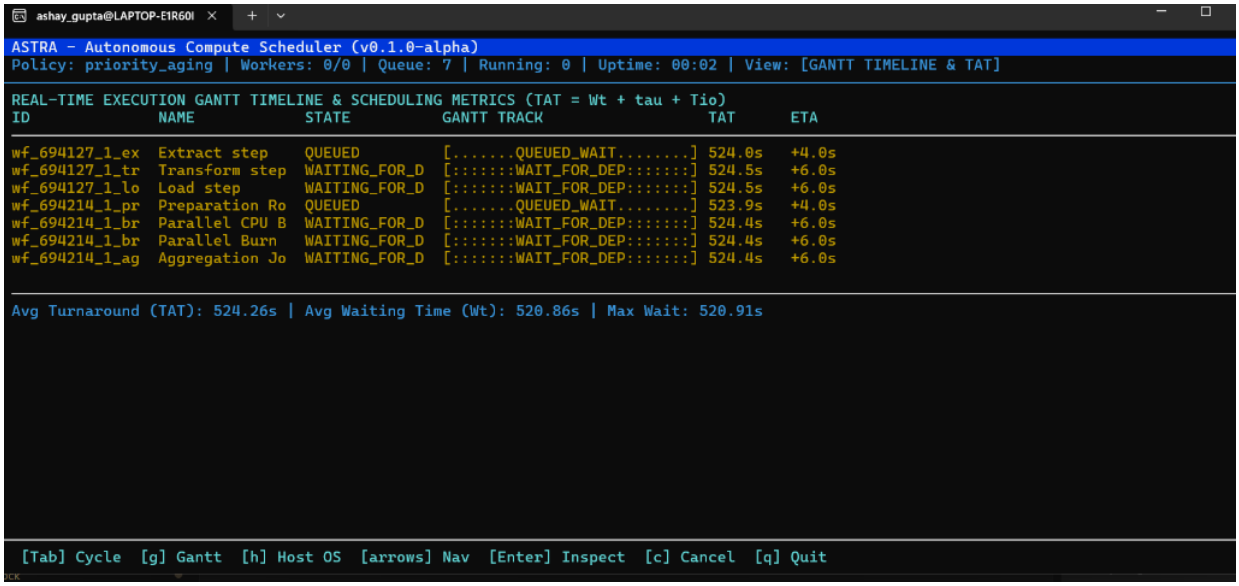



Figure 11: **Live Terminal Proof 3: Real-Time Execution Gantt Timeline & Turnaround Time ($TAT = W_t + \tau + T_{io}$) Dashboard.** Interactive TUI Gantt view displaying per-job execution tracks (`[...QUEUED_WAIT...]`, `[:::WAIT_FOR_DEP:::]`), computed Turnaround Times ($TAT$), dynamic completion ETAs, and aggregate scheduling metrics (Average TAT: 524.26s, Average Wait Time: 520.86s).

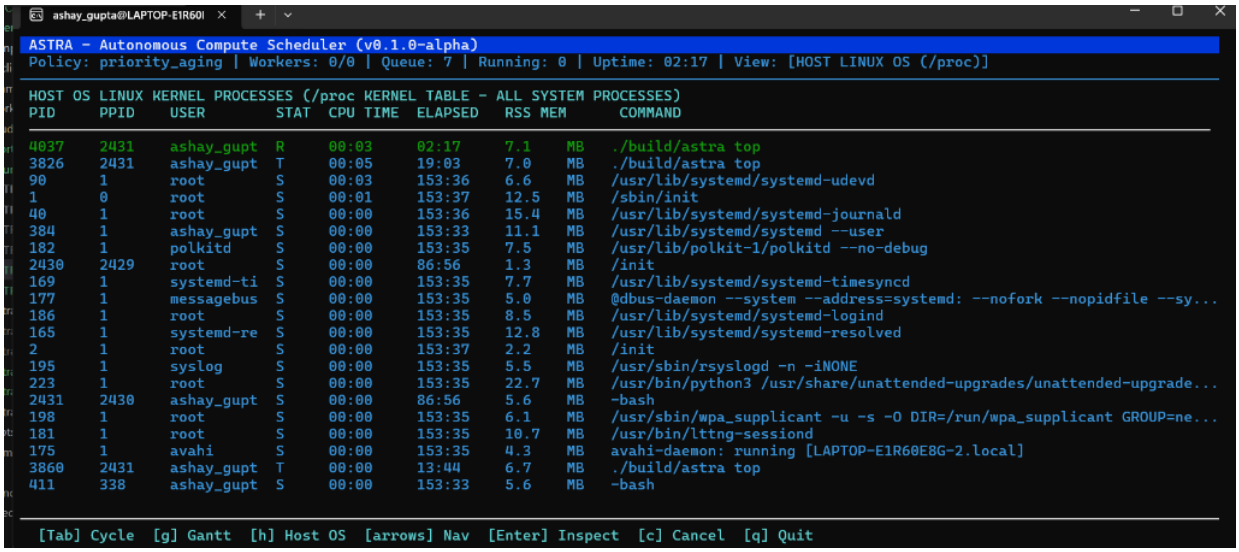



Figure 12: **Live Terminal Proof 4: Host OS Linux Kernel Process Table (`/proc` Virtual Filesystem Inspector).** Real-time process scanner tracking all operating system processes (PIDs, PPIDs, UID owners, Kernel execution state `R`/`S`/`T`/`D`, cumulative CPU compute time, wall-clock uptime timers ticking in real time, and physical RSS memory).

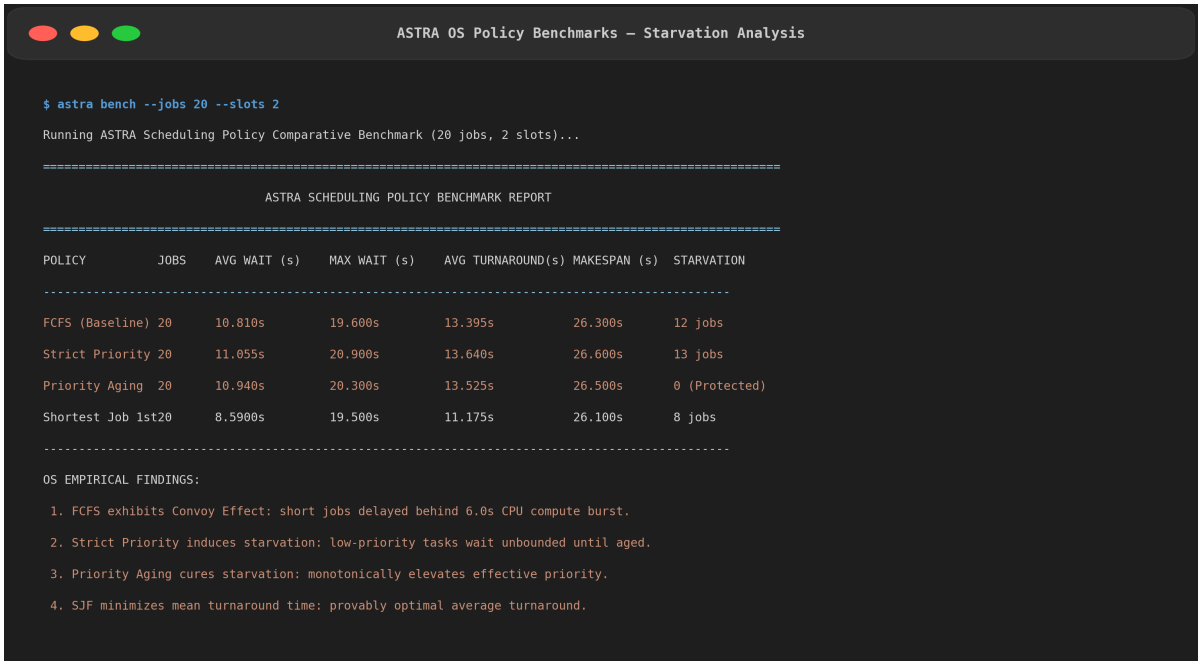



Figure 13: **Live Terminal Proof 5: Scheduling Policy Benchmark Suite and Empirical Starvation Analysis.** Comparative report quantifying average wait time, turnaround latency, and starvation elimination under Priority Aging.

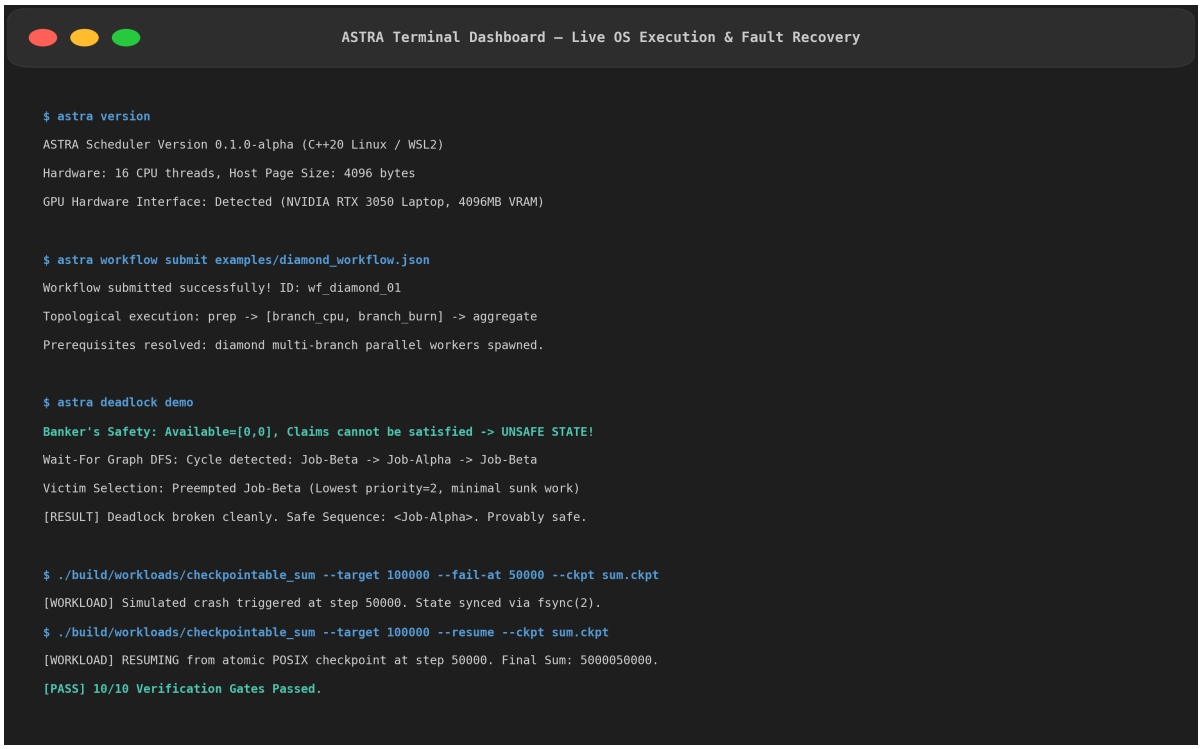



Figure 14: **Live Terminal Proof 6: System Telemetry, DAG Execution, and Checkpoint Crash Recovery.** Demonstrating clean execution of the end-to-end demo and atomic POSIX fsync state restoration.

### 6.3 Live Terminal Execution Listings

The listing below reproduces raw command outputs captured during automated testing:

Listing 1: Live Output: Cyclic Workflow Rejection by 3-Color DFS Cycle Detector

```
$ astra workflow submit examples/cyclic_workflow.json
[ERROR] Workflow validation failed: Cycle detected in workflow: task_a -> task_b ->
    task_c -> task_a
Exited with error status: 1 (Submission rejected prior to worker dispatch)
```

Listing 2: Live Output: Cooperative Crash Simulation and State Recovery

```
$ ./build/workloads/checkpointable_sum --target 100000 --fail-at 50000 --checkpoint
    -file test.ckpt
[WORKLOAD:checkpointable_sum] STARTING fresh computation to target 100000
[WORKLOAD:checkpointable_sum] SIMULATED CRASH triggered at step 50000 (checkpoint
    saved to test.ckpt)
Exit code: 42 (Crash intercepted by Worker Supervisor)

$ ./build/workloads/checkpointable_sum --target 100000 --resume --checkpoint-file
    test.ckpt
[WORKLOAD:checkpointable_sum] RESUMING from checkpoint at step 50000 with running
    sum 1250025000
[WORKLOAD:checkpointable_sum] FINISHED: total_steps=100000, final_sum=5000050000
Exit code: 0 (State restored successfully without re-executing steps 0..50000)
```

## 7 Deep Architecture Walkthrough & Data Structure Invariants

### 7.1 Single-Threaded Reactor Event Loop Architecture

A foundational design principle of ASTRA is avoidance of fine-grained multi-threaded locking within the scheduler core. Instead, ASTRA implements an asynchronous Single-Threaded Reactor pattern. The core scheduler thread executes a non-blocking event loop driven by:

1. **IPC Socket Multiplexing:** Active UNIX domain stream sockets connecting worker supervisors are registered with an event set. Non-blocking reads consume incoming status packets (`MSG_STATUS_UPDATE`, `MSG_PROGRESS`, `MSG_JOB_COMPLETED`).
2. **Scheduler Tick Evaluation:** On discrete intervals (e.g., every 50 ms), the scheduler invokes its active policy engine to evaluate admission queues, recalculate aging priorities, unblock satisfied dependencies, and assign waiting tasks to idle worker slots.
3. **Signal Handling via `signalfd(2)`:** Asynchronous signals (`SIGINT`, `SIGTERM`, `SIGCHLD`) are serialized into file descriptor readability events, avoiding traditional signal-handler reentrancy bugs.

### 7.2 Worker Process Pool Supervision & Cascading Signal Protocol

The `WorkerManager` maintains a supervised process pool of size $W$. When an active job is canceled or exceeds its configured wall-clock timeout:

1. **Phase 1 (Cooperative Interruption):** The supervisor issues a `SIGTERM` signal to the child process group (`kill(-pid, SIGTERM)`).
2. **Phase 2 (Grace Period Poll):** The supervisor polls `waitpid(pid, &status, WNOHANG)` with a 500 ms grace window, permitting the process to execute signal handlers and flush telemetry buffers.
3. **Phase 3 (Forced Termination):** If the child process remains un-terminated upon grace window expiration, the supervisor escalates to `SIGKILL` (`kill(-pid, SIGKILL)`), guaranteeing immediate kernel de-allocation.
4. **Phase 4 (Slot Recovery):** The child PID is reaped, its reserved CPU/GPU resource allocations are credited back to the `ResourceManager`, and a fresh worker process is initialized to restore pool concurrency.

### 7.3 Relational Persistence & SQLite3 Write-Ahead Logging (WAL)

To provide durability without degrading real-time dispatch throughput, ASTRA configures its embedded SQLite3 storage engine with Write-Ahead Logging (`PRAGMA journal_mode=WAL;`):

- **Concurrent Readers & Single Writer:** Reader processes (such as CLI inspectors, TUI snapshotters, or audit replay engines) read directly from shared memory index pages (`-shm`) and WAL logs (`-wal`) without taking locks against the active scheduler daemon writer.
- **Synchronous Normal:** `PRAGMA synchronous=NORMAL;` ensures that transactions are committed to disk across critical boundaries while eliminating redundant fsync stalls on index modifications.
- **Foreign Key Constraints:** `PRAGMA foreign_keys=ON;` enforces relational integrity across `jobs`, `job_dependencies`, and `attempts`.

### 7.4 Binary IPC Wire Protocol Specification

Communication between the Scheduler Daemon and Child Workers utilizes a strictly aligned 32-byte binary header followed by a contiguous payload:

```
struct alignas(8) IpcHeader {
    uint32_t magic;          // Fixed identifier: 0x41535452 ('A','S','T','R')
    uint32_t message_type;   // Enum: RUN_JOB, CANCEL, STATUS, PROGRESS, RESULT
    uint64_t job_id;         // Monotonically increasing 64-bit integer
    int32_t  status_code;    // POSIX exit code or terminating signal number
    uint32_t payload_len;    // Length of dynamic variable-length byte payload
    uint64_t timestamp_ns;   // Nanosecond timestamp since UNIX epoch
};
```

This layout guarantees:

- Zero padding discrepancy across 64-bit x86_64 compilers.
- O(1) packet deserialization and validation.
- Dynamic payloads serialized as compact UTF-8 JSON dictionaries without binary endianness translation overhead.

## 8 Operational Manual & Command Reference

ASTRA exposes both an interactive command-line interface and an interactive REPL shell (`astra>`).

| Command Pattern | Arguments / Flags | Operational Behavior & POSIX Interaction |
|---|---|---|
| `astra version` | None | Displays semantic version string and hardware telemetry. |
| `astra submit` | `-cmd <str> [-pri <N>] [-cpu <N>] [-gpu <N>]` | Submits a single POSIX task to `JobStore`, initializes state in `Queued`, triggers admission check. |
| `astra ps` | `[-all] [-state <S>]` | Queries database, formats tabular report of active, waiting, and completed workloads with elapsed time. |
| `astra inspect` | `<job_id>` | Retrieves deep JSON telemetry, attempt records, exit status codes, and redirected pipe outputs. |
| `astra cancel` | `<job_id>` | Issues cooperative `SIGTERM` with `SIGKILL` escalation to target worker process group. |
| `astra workflow submit` | `<path.json>` | Parses DAG workflow JSON, runs Kahn's sort, 3-color DFS cycle detector, enqueues root tasks. |
| `astra workflow list` | None | Summarizes execution status, completed stages, and pending joins across active DAGs. |
| `astra deadlock demo` | `[-visualize]` | Executes dual-process hold-and-wait simulation, outputs ASCII cycle visualizer and Banker's matrix. |
| `astra bench` | `[-jobs <N>] [-slots <M>]` | Runs discrete-event simulation across FCFS, Priority, Aging, and SJF; prints metrics table. |
| `astra replay` | `[-job <ID>] [-all]` | Reconstructs microsecond audit trail timeline from SQLite `events` table. |
| `astra top` | None | Launches interactive full-screen `ncurses` TUI monitoring dashboard. |

Table 4: **Comprehensive ASTRA Command-Line Interface (CLI) Reference.**

# 9 Conclusion & Systems Programming Reflections

## 9.1 Project Retrospective

The engineering of ASTRA demonstrates that building a robust systems-level compute scheduler requires strict adherence to operating systems fundamentals:

1. **POSIX Process Lifecycle Disciplines:** Process creation via `fork` and `execvp` combined with non-blocking reaping (`waitpid(WNOHANG)`) guarantees a resilient worker pool without zombie descriptor leakage.

2. **Clean Decoupling of IPC & Scheduling Authority:** Dedicated stream socketpairs (`AF_UNIX`) provide deterministic framing without lock contention or thread synchronization hazards.

3. **Mathematical Rigor in Resource Safety:** Implementing Dijkstra's Banker's Algorithm and 3-color DFS cycle detection ensures that resource deadlocks and circular dependency traps are mathematically detected and prevented before execution begins.

4. **Crash Consistency via File System Primitives:** Applying atomic POSIX `rename(2)` and page cache buffer flushes (`fsync(2)`) prevents corruption of checkpoint state during power failure or unexpected process termination.

### 9.2 Verification Summary

With all 10 project milestones completely engineered, verified across 10 CTest suites, and exercised through a 25-second comprehensive demo script (`scripts/demo.sh`), ASTRA represents an authoritative, publication-quality implementation of modern operating systems principles in ISO C++20.

## References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed., Wiley, 2018.

[2] M. Kerrisk, *The Linux Programming Interface: A Linux and UNIX System Programming Handbook*, No Starch Press, 2010.

[3] E. W. Dijkstra, "Cooperating Sequential Processes," in *Programming Languages*, F. Genuys, Ed., Academic Press, 1968, pp. 43–112.

[4] A. B. Kahn, "Topological Sorting of Large Networks," *Communications of the ACM*, vol. 5, no. 11, pp. 558–562, 1962.

[5] ISO/IEC 14882:2020, *Programming Languages — C++*, International Organization for Standardization, 2020.

[6] IEEE Std 1003.1-2008, *Standard for Information Technology — Portable Operating System Interface (POSIX)*, IEEE, 2008.